ADVANCED PROGRAMMING | ASSESSMENT 2

CSV Data Quality Lab

Technical documentation and evaluation - working draft

A desktop CSV profiling tool built with Python and PyQt6.

Student: [Insert student's own name and ID]

Module: Advanced Programming

Project type: Individual practical programming project

Status: SOURCE TESTS COMPLETED; Windows GUI and distribution evidence to be added

Evidence warning: The automated tests and benchmark included here were executed in the project-authoring environment. They are not evidence that the student has run the application or personally completed the Git workflow. Replace pending entries with the student's actual results before submission.

1. Project overview and requirements (Task 1)

CSV files are common in small business and classroom data exercises, but they may contain empty fields, repeated rows or inconsistent columns. CSV Data Quality Lab checks these basic issues in several local CSV files. Its purpose is not to replace specialist analytics software. It provides a simple overview so a user can identify where a dataset needs attention.

The intended users are students or office staff who need to inspect structured records. The application loads CSV files, analyzes each file without changing its contents, shows a bounded sample, profiles columns, and exports an independent cleaned copy.

ID	Functional requirement
FR01	Add one or more UTF-8 CSV datasets
FR02	Run inspection in a background worker
FR03	Count rows, blank cells and exact duplicates
FR04	Detect numeric columns and calculate statistics
FR05	Preview and filter up to 150 rows
FR06	Show progress and readable processing errors
FR07	Cancel an active batch safely
FR08	Export a cleaned copy without overwriting source
FR09	Export a data-quality summary
FR10	Play an audio completion notification

A text-based initial GUI wireframe is included in **docs/01_Requirements_Design.md**. The dashboard presents import and export controls in a left sidebar, four quality metrics above a tabbed workspace, and a progress bar below.

2. GUI implementation and events (Tasks 2 and 3)

The main window uses QMainWindow. Its main widgets include QListWidget for datasets, QTabWidget for different views, QTableWidget for preview and column profiles, QLineEdit for searching, QProgressBar for feedback, QMessageBox for results/errors, and QFileDialog for safe file selection. The File menu offers the same core workflows. A dark navy design and turquoise actions make the main analysis button easy to find.

Signals and slots connect user actions to short controller methods. For example, pressing Analyze All Datasets creates a worker instead of parsing CSV rows in the button callback. Validation rejects missing files, non-CSV extensions, repeated headings and inconsistent row widths. Errors are collected in the Activity log rather than silently ignored.

3. Separation of concerns (Task 4)

Component	Responsibility
analyser.py	Pure Python rules, immutable report models, source validation and safe exports.
worker.py	Runs CSV profiling on a QThread; sends result/progress/error signals.
gui.py	Renders views, handles buttons and coordinates results in the main thread.
main.py	Simple application launcher.

This is a separation-of-concerns approach with model, presentation and coordinating code. A completely separate controller class is not essential for a small Qt application, provided the interface does not contain the data-processing algorithms.

4. Multimedia integration (Task 5)

QtMultimedia QSoundEffect plays a short original WAV chime after a successful batch inspection. This is useful because a user can work elsewhere while a large dataset is processed. The sound is a confirmation, not decorative background music. It must be audibly tested on Windows before the report can claim this feature passes.

5. Concurrency and thread safety (Tasks 6 and 7)

CSV processing can take time when multiple tables have thousands of rows. BatchAnalysisWorker is a QObject moved to a QThread. The GUI does not wait on each file during normal operation and receives progress, errors and FileReport values using Qt signals. The worker holds an immutable tuple of input paths. The main thread alone updates the displayed reports and Qt widgets.

The only cross-thread cancellation state is a threading.Event, which supports safe set/is_set operations. Without this ownership discipline, direct worker updates to a QTableWidget could race with user interaction or crash. Cancellation is checked while reading records, allowing the current file operation to stop without partial result publication.

6. Performance investigation (Task 8)

The benchmark applies the same CSV profile function to a collection of six synthetic CSV files. It compares sequential execution, ThreadPoolExecutor and ProcessPoolExecutor, times each with time.perf_counter, repeats each three times and reports the median. The authoring environment used 2,500 data rows per generated file and four workers.

Approach	Median time	Relative speed-up
Sequential	0.1448 s	1.00x
Thread pool	0.2217 s	0.65x
Process pool	0.1598 s	0.91x

These are **actual authoring-environment measurements**, not the friend's Windows measurements. Here, sequential processing was fastest. Python thread scheduling, the interpreter lock, file access and process startup introduce overhead that can outweigh benefits for small tasks. Processes may help a larger CPU-bound workload; threads are more relevant to waiting on I/O. This experiment also illustrates parallel query-style work because separate files are analyzed independently and their results collected in a matching order.

Amdahl's Law example: If 80% of work could be parallelized across four workers, ideal maximum speed-up = 1 / [(1 - 0.80) + 0.80 / 4] = **2.50x**. This is a theoretical illustration, not the observed benchmark. The remaining sequential work and coordination overhead limit improvement.

7. Testing strategy (Tasks 9-11)

The test plan links cases to functional requirements and covers normal operation, boundary and exceptional inputs. Manual cases include opening files, searching, reviewing quality results, checking responsiveness and cancellation, verifying the confirmation sound, exporting a new CSV, and running the packaged executable.

The pure Python automated suite contains **35 tests** executed successfully in the authoring environment. They cover duplicate detection, missing values, numeric means, UTF-8 BOM support, malformed data, cancellation, output validation, cleaned exports, preserving original files and consistency of benchmark methods. Tests for visible Qt behaviour, audio and executable startup remain manual because they require a real desktop session.

Evidence	Current status
Python unittest suite	35 passed in authoring environment; friend must repeat
Windows GUI manual tests	PENDING - friend must execute and record
Windows audio and EXE tests	PENDING - friend must execute and record
Git history and conflict evidence	PENDING - friend must create and document

8. Debugging and corrective development (Task 12)

During development and testing, the student must record an issue that genuinely occurred, its likely cause, a corrective source-code change, and the result of rerunning the affected test. The defect log in the test-plan file is intentionally unfilled: inventing a defect or a successful retest would not count as genuine testing evidence.

9. Git and version control (Task 13)

The accompanying Git guide includes repository initialization, .gitignore, meaningful commits, a feature branch, merge, controlled conflict practice, and push/pull operations if allowed. Actual source-control evidence is not pre-filled as the friend's work. The student should use Git while reviewing and developing this separate project and capture the history they actually create.

10. Distribution (Task 14)

The included build_windows.bat uses PyInstaller --onefile --windowed and bundles assets/complete.wav. README provides installation and run instructions. A Windows EXE could not be built in the authoring environment. The friend should build it on a suitable Windows computer, run it without manually invoking Python files, and preserve screenshots and error logs.

11. Critical evaluation and future improvements (Task 15)

Strengths include a clear data-focused workflow, a bounded preview, reusable Python analysis routines, error messages, safe output files, cancellation and an independent benchmark. Limitations include UTF-8-only CSV support, exact rather than semantic duplicate detection, preview search that does not cover all rows, no database support, and memory used to retain duplicate row fingerprints. The quality percentage is a teaching indicator rather than a standardized quality measure.

Possible future improvements include encoding detection, progress estimates for individual large files, a streaming duplicate approximation for very large datasets, better per-column charts, and configurable cleaning rules. Before considering these improvements, priority should be given to authentic Windows verification, a clear presentation and final compliance with the assessment brief.

12. References (official technical sources)

Qt Documentation, *QThread Class*, https://doc.qt.io/qt-6/qthread.html

Qt Documentation, *QObject Class*, https://doc.qt.io/qt-6/qobject.html

Qt Documentation, *QSoundEffect Class*, https://doc.qt.io/qt-6/qsoundeffect.html

Python Software Foundation, *csv - CSV File Reading and Writing*, https://docs.python.org/3/library/csv.html

Python Software Foundation, *concurrent.futures - Launching parallel tasks*, https://docs.python.org/3/library/concurrent.futures.html

Python Software Foundation, *unittest - Unit testing framework*, https://docs.python.org/3/library/unittest.html

PyInstaller Development Team, *PyInstaller Manual*, https://pyinstaller.org/en/stable/

13. Submission evidence and in-class demonstration

Before submitting, add the student's real screenshots: dashboard after launch; imported datasets; quality metrics; searchable preview; summary export; automated test output; benchmark output; Git graph and conflict; packaged executable. Replace every PENDING status in the test plan with an observed result and explain any failures. Prepare to demonstrate a full analysis and explain background processing, thread safety, quality scoring and performance results in class.